# Requirement Analysis Skill

A deterministic pre-development planning engine for AI agents — given a free-text feature request, it identifies which files to change, the exact location, an existing pattern to follow, and the security, compliance, and formatting rules that apply. No Anthropic API key required.

---

Version 1.0.0
skill/requirement-analysis/

# Contents

# 1. Overview

The Requirement Analysis Skill is a polyglot agent skill for the Planning / Pre-development phase of the SDLC. Given a free-text feature or change request, it deterministically identifies which file(s) to touch, the exact function/class/line to change or add near, an existing implementation to use as a template, and the security, compliance, and formatting rules that apply. The host AI agent (Claude Code, GitHub Copilot, or Cursor) reads the engine's output and adds narrative judgment on top — no separate LLM API key or network call is required by the engine itself.

## What you get, every run

| # | Output | What it covers |
|---|---|---|
| 1 | Files To Update | Ranked candidate files, each with a match-reason and the exact symbol/line to change or add near |
| 2 | New File Suggestion | If no existing file fits, a suggested path and naming convention (only when the repo already has files of that type) |
| 3 | Pattern To Follow | An existing implementation of the same kind, as a structural template (for "add" actions) |
| 4 | Security & Compliance Checklist | Rule-based checklist keyed to the change type and domain, each item citing a standard (OWASP, PCI-DSS, GDPR, etc.) |
| 5 | Formatting & Linting | Detected formatter/linter plus the existing indent/quote/line-ending style of each target file |

## Why it exists

LLMs are great at writing code once you tell them where. They are much less reliable at finding where — especially in a large, unfamiliar codebase, where a guess can mean editing the wrong service, missing an existing convention, or skipping a security control every other endpoint already has. This skill is deterministic (same requirement text + same repo state = same output, byte-for-byte modulo the timestamp), explainable (every candidate lists why it was picked), and has zero third-party dependencies.

# 2. Requirements

| Tool | Version | Required |
|---|---|---|
| Python | 3.8 or later | Yes |
| Git | any | No — this skill does not use git at all |

No third-party packages. No API key. Fully offline.

# 3. Installation

Copy skill/requirement-analysis/ from this repository into your project's skills folder. The same folder contents work for all three tools — only the parent directory changes:

```
your-project/
  .claude/skills/requirement-analysis/     (Claude Code)
  .github/skills/requirement-analysis/     (GitHub Copilot)
  .cursor/skills/requirement-analysis/     (Cursor)
      SKILL.md
      manifest.yaml
      scripts/
      assets/
      references/
      templates/
```

## Verify Python

```
python --version
# Should print Python 3.8 or higher
```

Done — no pip install, no API key, no additional configuration.

# 4. Quick Usage

Ask your AI assistant in plain language. Mention concrete names (class names, field names, file names) if you know them — it sharply improves the match quality.

```
I want to add a discount_code field to the Order model
and expose it on the orders API
```

The assistant asks where to save the output (default ./requirement-analysis-output/), runs the Python engine, reads the JSON result, and presents a narrative plan with its own judgment on top.

## Output location options

| Option | Flag | Behavior |
|---|---|---|
| 1 (default) | (none) | ./requirement-analysis-output/ |
| 2 | --output . | Directly in the current directory |
| 3 | --output <path> | A specific path you provide |
| 4 | --json-only | Print JSON to stdout; write nothing to disk |

# 5. Command-Line Reference

The engine can also be run directly, without an AI agent:

```
python skill/requirement-analysis/scripts/requirement_analysis_skill.py \
  --requirement "<free text>" [--repo-path <path>] [--output <path>]
  [--top-n <N>] [--json-only]
```

| Argument | Default | Meaning |
|---|---|---|
| --repo-path | . | Root of the repository to analyze |
| --requirement | (required) | Free-text description of the feature/change |
| --top-n | 5 | Maximum number of candidate files returned |
| --output | <repo>/requirement-analysis-output/ | Output directory for markdown + JSON |
| --json-only | off | Print JSON to stdout, write nothing to disk |

## Exit codes

| Code | Meaning |
|---|---|
| 0 | Success |
| 1 | --repo-path does not exist |

## Examples

```
# Custom repo path and output directory
python scripts/requirement_analysis_skill.py \
  --repo-path /path/to/repo \
  --requirement "Fix the login endpoint to reject expired tokens" \
  --output ./reports/

# JSON only (stdout) -- pipe into other tooling
python scripts/requirement_analysis_skill.py \
  --requirement "Add a dark mode toggle to the settings page" --json-only

# Return more candidate files (default: 5)
python scripts/requirement_analysis_skill.py \
  --requirement "Add rate limiting to all public API endpoints" --top-n 10
```

# 6. Output Files Reference

| File | Description |
|---|---|
| implementation_plan.md | Primary artifact — requirement, parsed intent, files to update, new-file suggestion, pattern to follow, security/compliance checklist, formatting notes, next steps |
| requirement_analysis.json | Machine-readable result — identical data as structured JSON, for CI/CD or further tooling |

## requirement_analysis.json schema

```
{
  "repo_path": "string",
  "intent": {
    "raw_text": "...",
    "action": "add|modify|remove|fix",
    "target_types": ["api_endpoint", "database", ...],
    "entities": ["OrderService", "discount_code"],
    "domain_tags": ["financial_critical", ...]
  },
  "location": {
    "candidates": [
      {"path": "...", "module_type": "...", "score": 28,
       "match_reasons": [...], "matched_symbols": [...],
       "suggestion": "..."}
    ],
    "new_file_suggestion": null
  },
  "pattern": {"reference_file": "...", "reference_symbol": {...}, "note": "..."} | null,
  "security_compliance": [{"category": "security", "item": "...", "reference": "..."}],
  "formatting": {
    "detected_tools": [{"config_file": "...", "tool": "...", "command": "..."}],
    "candidate_file_styles": {"path": {"indent": "...", "quote_style": "...", "line_ending
": "..."}}
  }
}
```

A real, unedited example of both files is in marketplace/SampleOutputs/sample-app-report/.

# 7. Security & Compliance Rules Reference

Each requirement's checklist is the union of rules from four sources: the detected target type(s), the detected domain tag(s), the action (remove/fix only), and a fixed baseline — deduplicated and sorted security → compliance → standard.

## Target-type rules (excerpt)

| Target type | Category | Item | Reference |
|---|---|---|---|
| api_endpoint | security | Validate and sanitize all input parameters, query strings, and request bodies. | OWASP ASVS V5 |
| api_endpoint | security | Apply rate limiting / throttling to the new or changed endpoint. | OWASP API Top 10 API4 |
| database | security | Use parameterized queries / ORM methods only. | OWASP A03:2021 |
| database | compliance | Add a forward/backward-compatible migration; never edit an applied one. | Schema governance |
| ui_component | security | Escape/encode user-supplied data before rendering (prevent XSS). | OWASP A03:2021 |
| config | security | Never hardcode secrets — use env vars or a secrets manager. | OWASP ASVS V14 |

## Domain-tag rules (excerpt)

| Domain tag | Category | Item | Reference |
|---|---|---|---|
| security_sensitive | security | Never log raw passwords, tokens, or session identifiers. | OWASP ASVS V7/V8 |
| financial_critical | compliance | Never store raw payment card data — use a tokenized reference. | PCI-DSS |
| pii_data | compliance | Minimize personal data collected; encrypt sensitive fields at rest. | GDPR Art. 5/25 |
| external_integration | security | Verify signatures on inbound webhooks; use HTTPS outbound. | OWASP ASVS V13 |

## Baseline rules (always included)

- Add or update unit tests for the change.
- Run the project's formatter and linter before committing.
- Update relevant documentation, README, or changelog entries.

Full table with every rule and citation: references/security-compliance-rules.md.

# 8. How the Pipeline Works

```
requirement_analysis_skill.py  (CLI entry point)
        |
  [1] RequirementParser        -- free text -> action, types, entities, tags
  [2] CodebaseIndexer          -- walks repo once -> per-file symbols/tokens
  [3] LocationResolver         -- additive scoring -> ranked candidate files
  [4] PatternFinder            -- (add actions) finds an existing template
  [5] SecurityComplianceChecker -- rule-table union -> deduped checklist
  [6] FormattingAnalyzer       -- detects formatter/linter + per-file style
  [7] PlanGenerator            -- renders implementation_plan.md + JSON
```

Every stage is pure Python standard library, deterministic, and runs entirely offline — there is no LLM call anywhere in the engine. The host AI agent only reads the final output and adds narrative. See Architecture.png for the visual pipeline diagram.

## Determinism guarantees

- No randomness, no network calls, no LLM calls inside the engine.
- All dict/set iteration that affects output order is explicitly sorted.
- File-walk order does not affect the final ranking — scores are computed per file and the result list is fully re-sorted.

## Known v1 limitations

- No new-file suggestion if the repo has zero files of the requested module type (by design — won't fabricate a convention).
- Entity extraction is regex-based; multi-word natural-language names aren't extracted — quote them or use identifier-style names.
- PatternFinder only runs for action == "add" with a target type that has a template mapping (api_endpoint, database, service, ui_component).

# 9. Repository Layout

```
Requirement-analysis-skill/
  skill/
    requirement-analysis/     <- copy this into .claude/skills/
      SKILL.md
      manifest.yaml
      scripts/
        requirement_analysis_skill.py
        engine/
      assets/
      references/
      templates/
  marketplace/                 <- this folder - docs, guide, samples
    README.md
    QuickStart.md
    UserGuide.pdf                (this document)
    Architecture.png
    SamplePrompts.md
    SampleOutputs/
```

# 10. Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| "python: command not found" | Python 3.8+ not installed or not on PATH | Install Python 3.8+ and re-open your terminal / editor |
| Script not found at the expected path | Skill folder copied to the wrong location | Re-check Installation — the path must end in .../requirement-analysis/scripts/requirement_analysis_skill.py |
| Every candidate scores 0 or very low | Requirement text has no concrete identifiers the indexer can match | Add real class/field/file names, or quote them, and re-run |
| No new_file_suggestion even though no candidate fits | Repo has zero files of the detected target_type | By design — provide an explicit target directory or add a seed file of that type first |
| Output not written anywhere | --json-only flag was passed | Omit --json-only, or read the JSON printed to stdout directly |
| Output re-running produces a different candidate order | Repo files changed between runs | Expected — determinism holds only for identical requirement + identical repo state |

If Python or the script is unavailable, the host AI agent falls back to a manual workflow described in SKILL.md's "Manual Fallback" section — it uses file_search/grep_search directly and references/security-compliance-rules.md to build the checklist by hand.